Experiment 2 – Understanding Data with Programming

Dataset: urban_climate.csv

Experiment 2 – Understanding Data with Programming

Aim: To perform data handling and analysis using Python, Pandas and NumPy.

Dataset: `urban_climate(3).csv`

Instructions: Keep this notebook and the CSV file in the same folder. Run the cells from top to bottom.

1. Import libraries and load the CSV file

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("urban_climate(3).csv")
print("Dataset loaded successfully")
df.head()
```

Inference: The CSV file is loaded into a Pandas DataFrame. `head()` displays the first five rows.

2. Dataset information and description

```
print("Shape:", df.shape)
print("Columns:", df.columns)
df.info()

df.describe()
```

Inference: The dataset has 11,040 rows and 20 columns. It contains climate information such as temperature, humidity, rainfall and wind speed, along with city and country details.

3. Check and handle missing values

```
print(df.isnull().sum())

# Fill missing numerical values with their column average
df.fillna(df.mean(numeric_only=True), inplace=True)

# Fill missing text values with the most common value
for col in df.select_dtypes(include="object").columns:
    df[col].fillna(df[col].mode()[0], inplace=True)

print("Missing values after filling:")
print(df.isnull().sum().sum())
```

Inference: No missing values were found in the given CSV file. The code above shows how missing values can be filled if they are present.

4. Sort data and display top 5 or 8 records

```
# Top 5 highest temperature records
df.sort_values("temperature_celsius", ascending=False).head(5)

# Top 8 highest UHI intensity records
df.sort_values("urban_heat_island_intensity", ascending=False).head(8)
```

Inference: Sorting arranges records in order. These commands display records with the highest temperature and highest urban heat island intensity.

5. Frequency count (two columns)

```
print(df["city"].value_counts())

print(df["continent"].value_counts())
```

Inference: Frequency count tells us how many records are present for each city and continent.

6. Sorting and indexing

```
# Sort rows by city name
df.sort_values("city").head()

# Sort column names alphabetically
df[sorted(df.columns)].head()

# iloc uses row and column positions
print(df.iloc[0, 0])

# loc uses row and column labels
print(df.loc[0, "city"])
```

Inference: `sort_values()` sorts the rows. `iloc` selects data by position, while `loc` selects data using labels.

7. Conditional filtering (three cases)

```
# Case 1: Temperature above 25 and humidity above 60
df[(df["temperature_celsius"] > 25) & (df["humidity_percent"] > 60)].head()

# Case 2: Cities in Asia with rainfall above 100 mm
df[(df["continent"] == "Asia") & (df["precipitation_mm"] > 100)].head()

# Case 3: Records after 2015 with wind speed above 10
df[(df["year"] > 2015) & (df["wind_speed_ms"] > 10)].head()
```

Inference: Filtering displays only rows that satisfy the given conditions. The `&` operator means both conditions must be true.

8. Minimum and maximum values

```
print("Minimum values:")
print(df[["temperature_celsius", "humidity_percent", "precipitation_mm",
          "wind_speed_ms", "urban_heat_island_intensity"]].min())

print("Maximum values:")
print(df[["temperature_celsius", "humidity_percent", "precipitation_mm",
          "wind_speed_ms", "urban_heat_island_intensity"]].max())
```

Inference: Minimum and maximum values show the lowest and highest recorded values for each climate measurement.

9. GroupBy (two cases)

```
# Average temperature for each continent
df.groupby("continent")["temperature_celsius"].mean()

# Average humidity for each country and city
df.groupby(["country", "city"])["humidity_percent"].mean().head(10)
```

Inference: GroupBy divides the data into groups. We can compare average temperature by continent and average humidity by city.

10. Add a new column

```
# Convert temperature from Celsius to Fahrenheit
df["temperature_fahrenheit"] = df["temperature_celsius"] * 9/5 + 32

df[["city", "temperature_celsius", "temperature_fahrenheit"]].head()
```

Inference: A new column called `temperature_fahrenheit` is created using the existing Celsius temperature column.

11. Aggregate functions with GroupBy (two cases)

```
# Average and maximum temperature for each continent
df.groupby("continent")["temperature_celsius"].agg(["mean", "max"])

# Average humidity and total rainfall for each country
df.groupby("country").agg({
    "humidity_percent": "mean",
    "precipitation_mm": "sum"
})
```

Inference: Aggregation performs calculations on groups. Here, mean and maximum temperature are found for each continent, and average humidity and total rainfall are found for each country.

12. Select groups by name or condition

```
# Select records from Asia
asia = df[df["continent"] == "Asia"]
asia.head()

# Select continents where average temperature is above 15
avg_temp = df.groupby("continent")["temperature_celsius"].mean()
avg_temp[avg_temp > 15]
```

Inference: We can select a group by its name or select groups that satisfy a condition.

13. Correlation between two columns

```
print("Correlation between temperature and humidity:")
print(df["temperature_celsius"].corr(df["humidity_percent"]))
```

Inference: Correlation shows how two numerical columns are related. A positive value indicates that they tend to increase together, and a negative value indicates that one tends to decrease when the other increases. Correlation does not prove cause and effect.

14. Normalization

```
# Min-Max normalization changes values to a range between 0 and 1
minimum = df["temperature_celsius"].min()
maximum = df["temperature_celsius"].max()

df["temperature_normalized"] = (
    (df["temperature_celsius"] - minimum) / (maximum - minimum)
)

df[["temperature_celsius", "temperature_normalized"]].head()
```

Inference: Normalization scales the temperature values between 0 and 1. This is useful when data columns have different ranges.

15. Joining, merging and concatenation

```
# Make two small tables from the dataset
city_table = df[["city", "country"]].drop_duplicates()
temp_table = df.groupby("city")["temperature_celsius"].mean().reset_index()

# Merge using the city column
merged = pd.merge(city_table, temp_table, on="city")
merged.head()

# Join using city as the index
table1 = city_table.set_index("city")
table2 = temp_table.set_index("city")

joined = table1.join(table2)
joined.head()
```

```
# Concatenate two parts of the dataset
part1 = df.head(5)
part2 = df.iloc[5:10]

combined = pd.concat([part1, part2])
combined
```

Inference: Merge combines tables using a common column. Join combines tables using their index. Concatenation places DataFrames together row-wise or column-wise.

16. Simple histogram

```
df["temperature_celsius"].plot(kind="hist", bins=15)
plt.title("Temperature Distribution")
plt.xlabel("Temperature in Celsius")
plt.ylabel("Number of Records")
plt.show()
```

Inference: The histogram shows the distribution of temperature values. It helps us see which temperature ranges occur more frequently.

Post-lab questions

1. What is Broadcasting in NumPy?

Broadcasting allows NumPy to perform operations on arrays of different but compatible shapes.

Example:

```python
a = np.array([1, 2, 3])
print(a + 5)
```

Output: `[6 7 8]`

Here, 5 is added to every element of the array.

2. Difference between `loc[]` and `iloc[]`

loc[]	iloc[]
Label-based indexing	Position-based indexing
Uses row and column labels	Uses integer positions
`df.loc[0, "city"]`	`df.iloc[0, 0]`

Overall observations

1. The dataset contains 11,040 rows and 20 columns.
2. It has climate records for 20 cities in 10 countries.
3. The data covers 1980 to 2025.
4. No missing values were found.
5. Sorting and filtering help find records that match specific conditions.
6. GroupBy helps compare climate data for different cities, countries and continents.
7. Correlation helps study the relationship between numerical columns.
8. Normalization scales temperature values between 0 and 1.

9. Merge, join and concatenation are used to combine DataFrames.

Conclusion

In this experiment, I used Python, Pandas and NumPy to handle and analyse an urban climate dataset. I performed data exploration, missing value checking, sorting, filtering, grouping, aggregation, correlation and normalization. I also learned how to combine DataFrames using merge, join and concatenation.